Assignment 3

The concepts of words :

Question 1: What does `output.logits` represent?

`output.logits` represents the **unnormalized raw prediction scores (log-odds)** generated by the model for every single token in the vocabulary, across every position in your input sequence.

It is a 3D PyTorch Tensor with the shape: `[batch_size, sequence_length, vocab_size]`

- **`batch_size`**: The number of input text sequences processed simultaneously (in your notebook, this is `1`).
- **`sequence_length`**: The number of tokens in your input text sequence (e.g., `"The name of the president of USA is"` contains 8 tokens).
- **`vocab_size`**: The total size of GPT-2's vocabulary (which is exactly `50257` unique tokens).

The word meanings :

Token : In the context of Natural Language Processing (NLP) and Large Language Models (like the GPT-2 model used in your assignment), a **token** is the fundamental unit of data processed by the model.

Instead of reading text word-by-word or character-by-character, language models break down raw text into smaller pieces called tokens.

🔍 How Tokenization Works

A token can be a whole word, a part of a word (sub-word), or even a single character (like a punctuation mark or space).

GPT-2 uses a tokenization method called **Byte-Pair Encoding (BPE)**. Here is how a single sentence is broken down into tokens:

- **"The"** $\rightarrow$ might be a single token.
- **"president"** $\rightarrow$ might be a single token.
- **"Murkowski"** $\rightarrow$ (from Exercise 2 of your notebook) is an uncommon word, so the tokenizer splits it into multiple sub-word tokens: [" Mur", "kowski"].

Think of a 3D tensor like a **book**:

1. The **rows** on a page are the 1st dimension.

2. The **columns** across a page are the 2nd dimension (Rows + Columns = a 2D Page).
3. The **stack of pages** bound together forms the 3rd dimension (Pages = a 3D Book).

unnormalized raw prediction scores (log-odds):

When the model processes your text prompt, its final layer calculates a score for every single word (token) in its vocabulary to decide which one should come next. These scores are what we call **logits**.

Here is a breakdown of what makes them "unnormalized," "raw," and "log-odds."

1. What makes them "Raw" and "Unnormalized"?

In deep learning, **unnormalized** means the numbers haven't been scaled to fit a standard, constrained range (like percentages between 0% and 100%).

If you look closely at your assignment notebook's output for `output.logits`, the numbers look like this:

```
[-99.5066, -98.7923, -102.3320, ...]
```

- **No fixed bounds:** These scores can be *any* real number—ranging from negative infinity to positive infinity.
- **No standard sum:** They don't add up to anything meaningful. One token might have a score of `15.2`, another `-4.3`, and another `102.1`.

Because they are raw, you can't immediately use them as probabilities. For example, if Word A has a score of `10` and Word B has a score of `5`, it doesn't mean Word A is exactly twice as likely to appear as Word B. It just means Word A is significantly *more* likely.

2. What are "Log-Odds"?

In mathematics and statistics, **logits** are technically the **natural logarithms of the odds** (log-odds) of a token being the correct next choice.

If we break down the math step-by-step, the relationship between these raw scores and real-world probabilities looks like this:

1. **Probability (P):** The actual chance (between 0 and 1) that a token is next.
2. **Odds ($\frac{P}{1-P}$):** The ratio of the probability that the token *is* next to the probability that it *is not*. (For example, if a token has a 75% chance of being next, its odds are $\frac{0.75}{0.25} = 3$, or "3 to 1").
3. **Log-Odds ($\ln(\text{Odds})$):** Taking the natural logarithm of those odds. **This is your logit.**

3. How do we make them useful? (The Softmax step)

Because humans (and your random sampling code in Exercise 2) need actual percentages to make sense of choices, we have to transform these unnormalized log-odds into a clean probability distribution.

We do this using the **Softmax function** (F.softmax in your code):

$$\text{Probability}_i = \frac{e^{\text{logit}_i}}{\sum_{j} e^{\text{logit}_j}}$$
The Softmax function does two incredible things to the raw scores:

- It turns any negative or positive number into a positive decimal between **0 and 1**.
- It ensures that when you add up the scores of all 50,257 tokens in the vocabulary, **the sum is exactly 1.0 (100%)**.

Question 2: What does `output.logits[0, -1, :]` represent?

This specific slice isolates the **prediction scores for the next immediate token** following your current input sequence.

- `0`: Selects the first sequence in the batch.
- `-1`: Selects the **very last token position** in that sequence (using Python's negative indexing).
- `:`: Retains all `50257` raw vocabulary scores for that specific position.

💡 **Why is this important?** In autoregressive generation (like GPT-2), the model looks at all tokens in the prompt, but it only uses the prediction distribution at the *final* token position to choose what token comes next.

In exercise 6 :

What is "Temperature" in LLMs?

In deep learning, **temperature** is a hyperparameter borrowed from thermodynamics. It is used to scale the raw, unnormalized outputs of the model (**logits**) before they are turned into probabilities using the softmax function.

Effectively, temperature acts as a **"creativity" or "randomness" knob** for text generation:

- **Low Temperature ($T < 1.0$):** Makes the model more confident, conservative, and deterministic. It peaks the probabilities toward the most likely words and flattens out the less likely ones.
- **Default Temperature ($T = 1.0$):** The model uses its standard, calculated probabilities without any extra adjustment.
- **High Temperature ($T > 1.0$):** Makes the model more creative, random, or chaotic. It flattens the probability distribution, making less likely words much more likely to be picked.

Mathematical Meaning in Question 6

The assignment provides the formula for the temperature-adjusted softmax:

$$p_{i}=\frac{\exp(z_{i}/T)}{\sum_{j}\exp(z_{j}/T)}$$
Where:

- z_i represents the raw **logits** for a given token i generated by GPT-2.
- T is the **temperature** value (e.g., 0.7, 1.0, or 1.5).
- p_i is the final **probability** assigned to that token.

The process of exercises :

Exercise 1 – Greedy Generation

In the first exercise, I implemented greedy decoding for text generation. The objective was to complete the generate() function so that the model always selects the token with the highest prediction score at each generation step.

After passing the input text to GPT-2, the model produces a tensor called output.logits, which contains raw prediction scores for every token in the vocabulary. To generate the next token, I extracted the logits corresponding to the last position of the sequence using:

next_token_logits = output.logits[:, -1, :]

The next token was then selected using:

next_token = torch.argmax(next_token_logits, dim=-1).item()

The argmax operation chooses the token with the highest score. This process was repeated iteratively inside a loop, appending each newly generated token to the input sequence until a stop token or the maximum number of tokens was reached.

When generating text from the prompt:

"The name of the president of USA is"

the model produced:

"The name of the president of USA is "President Obama.""

This exercise demonstrated how autoregressive language models generate text by repeatedly selecting the most probable next token. Greedy decoding produces deterministic outputs and tends to generate coherent text, but it lacks diversity because it always chooses the single most likely option.

Exercise 2 – Random Sampling

In the second exercise, I extended the generation function to support probabilistic sampling. Instead of always selecting the highest-scoring token, the model was allowed to sample tokens according to their predicted probabilities.

First, the logits were converted into probabilities using the softmax function:

probs = F.softmax(next_token_logits, dim=-1)

Then, the next token was sampled from the probability distribution using:

next_token = torch.multinomial(probs, num_samples=1).item()

Unlike greedy decoding, this approach introduces randomness into the generation process. Tokens with higher probabilities are still more likely to be selected, but less probable tokens can also be chosen.

Using the same prompt, the generated output differed significantly from the greedy result. For example, the model generated text beginning with:

"The name of the president of USA is Warren Buffett..."

This demonstrates the increased diversity of random sampling. However, it also shows one of its disadvantages: the model may generate less accurate or less coherent continuations because it can sample from a very large vocabulary, including relatively unlikely tokens.

The main lesson from this exercise is that sampling increases creativity and variability but may reduce factual accuracy and consistency.

Exercise 3 – Top-k Sampling

In the third exercise, I implemented top-k sampling to improve the quality of probabilistic generation. The goal was to restrict sampling to only the k most likely candidate tokens before performing random selection.

The top-k candidates were obtained using:

```
topk = torch.topk(next_token_logits, k=top_k, dim=-1)
```

This function returns both the scores (topk.values) and the corresponding token indices (topk.indices) for the k highest-scoring tokens.

The probabilities were then computed only over these top-k candidates:

```
probs = F.softmax(topk.values, dim=-1)
```

A token was sampled from this reduced distribution:

```
sampled_idx = torch.multinomial(probs, num_samples=1)
```

Finally, the sampled position was mapped back to the original vocabulary index:

next_token = topk.indices.gather(-1, sampled_idx).item()

Using top_k = 10, the model generated:

"The name of the president of USA is "The Donald." He is a very conservative Republican..."

Although the generated statement is not necessarily factually correct, the output was generally more coherent than the unrestricted sampling used in Exercise 2.

Top-k sampling provides a balance between deterministic generation and unrestricted randomness. It preserves diversity by allowing multiple possible continuations while reducing the chance of selecting highly unlikely tokens. As a result, it often produces more natural and fluent text than standard sampling.

Conclusion

This assignment demonstrated three common text generation strategies used in autoregressive language models:

Greedy Decoding selects the most probable token at every step, producing deterministic and consistent outputs.

Random Sampling generates more diverse text by sampling from the entire probability distribution but may introduce errors and incoherent continuations.

Top-k Sampling restricts sampling to the most likely candidates, achieving a better balance between diversity and quality.

Through these exercises, I gained a practical understanding of how transformer-based language models generate text and how different decoding strategies affect the quality, diversity, and coherence of generated outputs.

Exercise 4 –:

Exercise 4 – Temperature-Based Sampling

In this exercise, temperature scaling was introduced into the token generation process. Temperature is a parameter that controls the randomness of the probability distribution used during sampling. Before applying the softmax function, the model logits were divided by a temperature value (T). The adjusted probabilities were then used to sample the next token.

Three different temperature values were tested:

Temperature = 0.7

When the temperature was set to 0.7, the probability distribution became sharper. High-probability tokens received even higher probabilities, while low-probability tokens became less likely to be selected. As a result, the generated text was more deterministic and closely followed the model's most likely continuation. The output was generally more coherent and predictable.

Temperature = 1.0

A temperature of 1.0 corresponds to the standard sampling procedure. The original probability distribution produced by the model was preserved without modification. The generated text showed a balance between coherence and diversity.

Temperature = 1.5

When the temperature was increased to 1.5, the probability distribution became flatter. Tokens with lower probabilities became more likely to be selected, introducing greater randomness into the generation process. The resulting text was more diverse and creative, although it occasionally became less coherent and less focused.

Observations

The experiments demonstrated that temperature directly affects the diversity of generated text. Lower temperatures encourage the model to select highly probable tokens, producing safer and more predictable outputs. Higher temperatures increase randomness and creativity but may reduce overall text quality and coherence.

The effect of temperature can therefore be viewed as a trade-off between determinism and diversity. Choosing an appropriate temperature depends on the desired application. Tasks requiring factual accuracy and consistency generally benefit from lower temperatures, whereas creative writing tasks may benefit from higher temperatures.

Question 3

Does it make sense to apply temperature scaling in greedy decoding?

No. Temperature scaling is generally not useful when using greedy decoding. In greedy decoding, the model always selects the token with the highest logit value using the argmax operation. Temperature modifies the probability distribution used for sampling, but it does not typically change the ranking of logits. Since greedy decoding does not sample from the distribution and only chooses the most probable token, changing the temperature has little or no practical effect on the generated output.

Therefore, temperature scaling is mainly relevant when random sampling is enabled and is not particularly meaningful in greedy decoding.

Conclusion

This assignment explored several token generation strategies for autoregressive language models. Starting from greedy decoding, the generation function was progressively extended to support random sampling, top-k sampling, and temperature-based sampling. The experiments showed how different decoding strategies influence the quality, diversity, and randomness of generated text.

Greedy decoding produced the most deterministic outputs, while random sampling introduced variability. Top-k sampling restricted generation to the most likely candidates, providing a balance between coherence and diversity. Finally, temperature scaling allowed direct control over the randomness of the token distribution. Lower temperatures generated more focused and predictable text, whereas higher temperatures increased diversity and creativity.

Overall, the assignment provided practical insight into the mechanisms used by modern language models to generate text and demonstrated how decoding strategies can significantly affect generation behavior.

Explaining the codes :

This Python code is using the **Hugging Face `transformers`** library and **PyTorch (`torch`)** to load a pre-trained AI language model (**GPT-2**) and prepare it to generate or analyze text.

- **`import torch`**: Loads PyTorch, the underlying machine learning framework that runs the mathematical calculations for the AI model.

- **`AutoTokenizer`**: A tool that breaks down human text into small pieces (tokens/numbers) that the AI can actually understand.

- **`AutoModelForCausalLM`**: The class used to load "causal language models"—models designed to predict the next word in a sentence (like GPT-2).

- **`import torch.nn.functional as F`**: Imports math functions (like Softmax) that are often used later to convert the model's raw scores into probabilities.

```
tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
```

- This downloads (or pulls from your local cache) the official configuration, vocabulary, and learned weights of the **GPT-2** model created by OpenAI.
- The `tokenizer` learns *how* GPT-2 splits words, and the `model` loads the massive brain of the AI.

```
text = 'The name of the president of USA is'
```

- This defines the **prompt** or seed text that you want to feed into the AI. In this case, it is an incomplete sentence: `"The name of the president of USA is"`.

```
input_ids = tokenizer(text, return_tensors='pt',
return_attention_mask=False)['input_ids']
output = model(input_ids=input_ids)
```

- **`tokenizer(...)`**: Translates your text string into a list of numbers (IDs) that represent those words. `return_tensors='pt'` tells it to format these numbers specifically for PyTorch.
- **`model(input_ids=input_ids)`**: Passes those numbers into the GPT-2 neural network.

1. Setting Up Stop Signs (`EOS_TOKENS`)

Python
```
EOS_TOKENS = [
    198, # newline
    50256 # tokenizer.eos_token_id = '<|endoftext|>'
]
```

An AI needs to know when to stop talking, or it will just keep rambling forever. This list defines the **End Of Sequence (EOS)** tokens.

- `198` is the ID for hitting the "Enter" key (a new line).
- `50256` is a special hidden token GPT-2 uses that literally means *"I am finished writing this thought."*
- `def generate(model, tokenizer, text, max_new_tokens=100, do_sample=False, top_k=None, verbose=True):`
- This sets up a reusable function. It accepts your model, tokenizer, and prompt (`text`). By default, it will generate up to `100` new tokens, and `verbose=True` means it will print the words out live as it thinks of them.
- # 3. The Core Generation Loop (`for i in range(...)`)
- This loop runs repeatedly up to `max_new_tokens` times. This strategy is called **Autoregressive Generation**—using past outputs as future inputs.
- ## Step A: Run the Model
- Python
- `output = model(input_ids = input_ids)`
- Just like in your first script, it passes the current sequence of numbers into GPT-2 to get its raw predictions.

Step B: Pick the Best Word (Greedy Search)

Python
```
next_token_logits = output.logits[:, -1, :]
next_token = torch.argmax(next_token_logits, dim=-1).item()
```

- `output.logits[:, -1, :]` looks strictly at the predictions for the **very last token** in the sequence.
- `torch.argmax(...)` finds the single word ID that got the highest math score from the model. This strategy is called "Greedy Decoding" because it always picks the absolute most likely next word without any randomness.

Step C: Glue the New Word onto the Input

Python
```
input_ids = torch.cat([input_ids, torch.tensor([[next_token]])], dim=1)
```

This is the magic step. It takes your original sequence (`input_ids`) and **concatenates** (glues) the `next_token` to the end of it.

- *Example:* If your input was *"The cat sat on the"*, and it guessed *"mat"*, the new input for the next round of the loop becomes *"The cat sat on the mat"*.
- **Step D: Check for Stop Signs**
- Python
- `if next_token in EOS_TOKENS:`
- `    break`
- If the word the AI just guessed happens to be a newline (`198`) or the end-of-text marker (`50256`), the loop hits `break` and stops immediately, even if it hasn't reached 100 words yet.

Codes in the Playground :

Instead of letting a loop do it automatically, you are executing the generation **step-by-step, one token at a time**, to see exactly how the AI appends words and updates its predictions.

This block sets up the environment and asks the model to predict the very first token following your prompt: `"The name of the president of USA is"`.

- **`output = model(input_ids = input_ids)`**: Passes your original prompt into GPT-2.
- **`next_token = torch.argmax(...)`**: Finds the highest-scoring token ID for the next word.
- **`print(tokenizer.decode([next_token]))`** *(in block 9)*: Converts that ID back into readable text.

As you can see from the output under block `[9]`, the model predicted a **quotation mark (`"`)** as the very next character.

So, your text sequence at this point effectively looks like:

```
"The name of the president of USA is \""
```

Step 3: Find and print the next word

Python

```
next_token_logits = output.logits[:, -1, :]
next_token = torch.argmax(next_token_logits, dim=-1).item()

print(tokenizer.decode([next_token]))
```

The model evaluates this new sentence, looks at the new end of the line, and calculates what should come next. As you can see from the printed output at the very bottom, it chose the word: **`President`**.

Ex2 :

The Key Code Difference: `if do_sample`

In your previous script, the code always picked the absolute highest-scoring word using `torch.argmax`.

In this new code, an `if/else` statement splits how the next word is selected:

Python
```
if do_sample:
    probs = F.softmax(next_token_logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1).item()
else:
    next_token = torch.argmax(next_token_logits, dim=-1).item()
```

1. The `else:` Path (Greedy Search)

If `do_sample=False`, it does exactly what your previous code did. It uses **Greedy Decoding** (`torch.argmax`), which always picks the #1 most probable token.

- **The Catch:** If you run the prompt 100 times, you will get the exact same answer 100 times. It can also get stuck in repetitive loops (e.g., *"and then and then and then"*).

2. The `if do_sample:` Path (Probabilistic Sampling)

If `do_sample=True` (which is being triggered in cell `[18]`), it switches to a two-step process to introduce randomness:

- **`F.softmax(next_token_logits, dim=-1)`**: Converts the model's raw math scores (logits) into clean percentage probabilities that all add up to 100%. For example, the next word options might look like: `{"Donald": 70%, "Barack": 20%, "a": 5%, "banana": 0.01%}`.
- **`torch.multinomial(probs, num_samples=1)`**: Think of this like spinning a roulette wheel where the sizes of the slots match those percentages. Because "Donald" has a 70% slot, it will *usually* win, but there is a 20% chance it picks "Barack", and even a tiny 0.01% chance it picks "banana".

Ex3 :

- **torch.topk(..., k=top_k)**: Instead of looking at all 50,000+ words the model knows, this cuts off everything except the top k highest-scoring words. If top_k=5, it grabs *only* the top 5 absolute best options.

- **F.softmax(topk.values)**: It scales the math scores into percentages *only* for those top 5 choices. The remaining 49,995 words are thrown in the trash and given a 0% chance.

- **The gather() step**: Because you isolated just the top 5 words, torch.multinomial returns an index from 0 to 4. The .gather() function maps that number back to the word's true ID in the main vocabulary.